

计算机系统导论（ICS）

Introduction to Computer System

Author: Simson

Date: September 2026

目录

1 基本介绍	1
1.1 ICS 的主要内容	1
1.2 CSAPP 的主要章节	1
2 信息的表示和处理	2
2.1 信息的二进制表示	2
2.1.1 信息量的单位	2
2.1.2 Boolean Algebra	2
2.1.3 数据的表示	2
2.2 C 语言的运算	3
2.2.1 算术运算	3
2.2.2 逻辑运算	3
2.2.3 移位运算	3
2.2.4 C 语言的计算	3
2.3 信息的表示和存储	4

1 基本介绍

本课程非常著名，使用的教材是 CSAPP 《深入理解计算机系统》

1.1 ICS 的主要内容

- 机器语言和编译（编译优化生成机器语言）
- 程序性能评估和优化
- 存储结构组织和管理
- 网络技术和协议
- 并行计算相关知识

1.2 CSAPP 的主要章节

1. 数据类型及其表示：了解计算规律和数学算术规律的差别
2. 汇编程序和程序的机器级表示
 - 了解程序执行模型，查找相关的程序错误（底层汇编代码工具）
 - 程序性能调优
 - 软件开发和系统软件
 - 安全性分析（汇编代码静态分析）
3. 处理器体系结构
4. 存储器层次结构：程序和数据在栈上的分布
5. 异常控制流（定制 shell 程序实验）
 - 进程控制
 - 信号和信号处理
6. 虚拟内存：数据的布局和数据组织
7. 算法性能分析
 - 内存性能影响程序性能
 - 程序编写符合缓存大小和性能
8. 计算机网络和并发编程（代理服务器）

2 信息的表示和处理

2.1 信息的二进制表示

Reason : Binary presentation is the most practical system to use.

2.1.1 信息量的单位

bits 是最基本的单位。

Byte = 8 bits

2.1.2 Boolean Algebra

逻辑运算的代数表示, 与, 或, 非, 异或。

用 boolean 代数的表示方法, 可以将与或非异或运算对应到交并补差等集合运算

2.1.3 数据的表示

Unsigned 数据 直接转换二进制存储

$$B2U(X) = \sum_{i=0}^{w-1} 2^i$$

范围:

$$Umin = 0, \quad Umax = 2^w - 1$$

Signed 数据 负数原来使用源码表示, 正负 0 不好区分等问题最终弃用, 改为补码(Two's Complement)。最高位为符号位, 0 为正数, 1 为负数

$$B2T(X) = -2^{w-1} + \sum_{i=0}^{w-2} 2^i$$

负数的补码等于模减去该负数的绝对值; 对于某一确定的模, 某数减去小于模的另一个数, 总可以用另一个数字加上该数字的补码来确定

范围:

$$Tmin = -2^{w-1}, \quad Tmax = 2^{w-1} - 1$$

Example

10 → 01010

-10 → 10110

-1 → 11111111...

(全是 1)

在编程中，有不同计算机的 long 类型有关键字可以跨平台进行。

```
1 #include <limits.h>
2
3 LONG_MAX
4 LONG_MIN
```

在 C 语言中，常量如果不加 U，则默认为有符号数。加 U 则认为无符号数。

类型的转换相当于先转换为二进制表示，再对于同一个二进制串使用不同的方式进行解读。在进行数值比较的时候，如果有无符号数同时出现，自动转换为无符号数字。

存储大小如果变大，在高位复制填入符号位直到填满为止，操作称为扩充。存储大小如果缩小，直接按照所需的位数进行溢出截断，相当于对 2^n 进行取模运算

2.2 C 语言的运算

C 语言中，将数据看作二进制向量进行处理。

- 算术运算
- 逻辑运算
- 移位运算

2.2.1 算术运算

和位相关，直接按位进行 Boolean 代数运算。

2.2.2 逻辑运算

主要是逻辑与，逻辑或

2.2.3 移位运算

左移：右侧补 0 右移：分算术右移和逻辑右移，逻辑右移直接向右移动高位补 0，算术右移右移补符号位

2.2.4 C 语言的计算

C 语言加法 首先是无符号数字的加法，首先是二进制加法，有可能发生溢出，相当于取模运算，最多取模一次

$$Uadd(u, v) = u + v \bmod 2^w$$

有符号数字的加法在 bit 位上有与无符号数完全相同的行为。如果计算结果超出了范围，可能发生正溢出，也可能发生负溢出。

$$\text{TAdd}_w(u, v) = \begin{cases} u + v + 2^w, & u + v < TMin_w \quad (\text{NegOver}) \\ u + v, & TMin_w \leq u + v \leq TMax_w \\ u + v - 2^w, & TMax_w < u + v \quad (\text{PosOver}) \end{cases}$$

C 语言乘法 乘法发生溢出的情况可能更多，可能需要算法包进行扩展，增大存储范围。

$$\text{Umult}(u, v) = uv \bmod 2^w$$

由于可能发生正负溢出，所以 TMult 有时候和 UMult 相比会出现不同的正负性。

乘法的具体操作是使用左右移动的方法，将乘数分解为二进制。左移 k 位，相当乘 2 的 k 次幂。

C 语言除法 除法相对于乘法，采用右移的方式来实现。除法涉及到向上取整的问题。

2.3 信息的表示和存储

内存就是一个大型的数组，以 word 为单位存储。程序通过地址访问内存中的程序。内存中最基本的单位是字，一个地址占据的空间大小称为字长；这个参数决定了地址在内存中的分布。系统会按照字节进行编址和寻址。常见的系统是 32-bit 或者 64-bit，指的是一个地址空间对应的比特位。

一个多字节的数据，例如一个整数，占据 4 个字节。根据多字节数据在内存中排布的字节顺序，决定了两种存储模式。大端法指的是：最小的有效位置放在最大的地址，反之小端法最小的有效数字放在最小地址（地址的标示是整数），需要注意的是 Internet 的存储方式是使用大端法。

每个字符只占据一个字节，所以字符串的存储不受到大小端法的影响，可以用字符串起到标示和检查的效果。